
TRÍ TUỆ NHÂN TẠO

Binary classification (Rice)

Bài tập Colab — Phân loại 2 giống gạo

Biên soạn:

ĐẶNG TẤN QUÍ

SĐT: 0377 122 210

2026

Mục lục

Lời nói đầu	3
1 Nạp thư viện và dữ liệu	4
1.1 Cài và import thư viện	4
1.2 Nạp dataset	4
2 Bài tập 1: Mô tả dữ liệu	5
3 Khám phá dữ liệu bằng trực quan hóa	6
4 Bài tập 2: Trực quan hóa 3D	6
5 Chuẩn hóa dữ liệu (Normalization)	8
6 Đặt seed, tạo nhãn và chia train/validation/test	8
7 Huấn luyện mô hình phân loại nhị phân	11
7.1 Chọn feature đầu vào	11
7.2 Định nghĩa hàm tạo và huấn luyện mô hình	11
7.3 Chạy thí nghiệm baseline và vẽ metric	12
7.4 Đánh giá trên validation	13
7.5 Huấn luyện mô hình dùng tất cả feature	14
7.6 Đánh giá mô hình đầy đủ trên validation	15
7.7 So sánh hai mô hình	15
8 Tính metric cuối trên test	17

Lời nói đầu

Nguồn

Tài liệu dịch từ notebook Colab *Binary classification* (ML Crash Course, 2023): https://colab.research.google.com/github/google/eng-edu/blob/main/ml/cc/exercises/binary_classification_rice.ipynb.

Bài toán

Bạn sẽ:

- Khảo sát dataset gồm các phép đo trích xuất từ ảnh của **hai giống gạo** Thổ Nhĩ Kỳ.
- Huấn luyện một bộ phân loại nhị phân để phân biệt hai giống.
- Đánh giá chất lượng mô hình.

Mục tiêu học tập

Bạn sẽ học:

- Cách huấn luyện bộ phân loại nhị phân.
- Cách tính các metric ở các ngưỡng (threshold) khác nhau.
- Cách so sánh ROC và AUC giữa hai mô hình.

Dataset

Bài tập dùng bộ dữ liệu gạo Osmancik và Cammeo (Cinar & Koklu, 2019), CC0. Tài liệu Kaggle: <https://www.kaggle.com/datasets/muratkokludataset/rice-dataset-commeo-and-osmancik>. Độ dài và diện tích được đo theo pixel. Tác giả cũng cung cấp nhiều dataset tương tự: <https://www.muratkoklu.com/datasets/>.

1 Nạp thư viện và dữ liệu

1.1 Cài và import thư viện

```
# @title Install required libraries

!pip install google-ml-edu==0.1.2 \
    keras~=3.8.0 \
    matplotlib~=3.10.0 \
    numpy~=2.0.0 \
    pandas~=2.2.0 \
    tensorflow~=2.18.0

print('\n\nAll requirements successfully installed.')
```

```
# @title Load the imports

import keras
import ml_edu.experiment
import ml_edu.results
import numpy as np
import pandas as pd
import plotly.express as px

# The following lines adjust the granularity of reporting.
pd.options.display.max_rows = 10
pd.options.display.float_format = "{:.1f}".format

print("Ran the import statements.")
```

1.2 Nạp dataset

```
# @title Load the dataset
rice_dataset_raw = pd.read_csv("https://download.mlcc.google.com/mledu-datasets/
    Rice_Cammeo_Osmancik.csv")
```

Chọn cột và xem thống kê mô tả

Sau khi nạp, hãy chọn một số cột để xem thống kê mô tả của các feature số. Giải thích ý nghĩa từng feature xem ở Kaggle (nhất là phần **Provenance**).

```
# Read and provide statistics on the dataset.
rice_dataset = rice_dataset_raw[[
    'Area',
    'Perimeter',
    'Major_Axis_Length',
    'Minor_Axis_Length',
    'Eccentricity',
    'Convex_Area',
    'Extent',
```

```
'Class',  
]]  
  
rice_dataset.describe()
```

2 Bài tập 1: Mô tả dữ liệu

Câu hỏi

Từ bảng thống kê, trả lời:

- Độ dài nhỏ nhất và lớn nhất (trục chính — `Major_Axis_Length`, đơn vị pixel) là bao nhiêu?
- Chênh lệch diện tích giữa hạt nhỏ nhất và lớn nhất là bao nhiêu?
- Chu vi (`Perimeter`) của hạt lớn nhất cách trung bình bao nhiêu độ lệch chuẩn (`std`)?

```
# @title Solutions (run the cell to get the answers)  
  
print(  
    f'The shortest grain is {rice_dataset.Major_Axis_Length.min():.1f}px long, '  
    f' while the longest is {rice_dataset.Major_Axis_Length.max():.1f}px. '  
)  
print(  
    f'The smallest rice grain has an area of {rice_dataset.Area.min()}px, while '  
    f' the largest has an area of {rice_dataset.Area.max()}px. '  
)  
print(  
    'The largest rice grain, with a perimeter of '  
    f' {rice_dataset.Perimeter.max():.1f}px, is '  
    f' ~{(rice_dataset.Perimeter.max() - rice_dataset.Perimeter.mean())/ '  
    rice_dataset.Perimeter.std():.1f} standard '  
    f' deviations ({rice_dataset.Perimeter.std():.1f}) from the mean '  
    f' ({rice_dataset.Perimeter.mean():.1f}px). '  
)  
print(  
    f'This is calculated as: ({rice_dataset.Perimeter.max():.1f} - '  
    f' {rice_dataset.Perimeter.mean():.1f})/{rice_dataset.Perimeter.std():.1f} = '  
    f' {(rice_dataset.Perimeter.max() - rice_dataset.Perimeter.mean())/ '  
    rice_dataset.Perimeter.std():.1f} '  
)
```

3 Khám phá dữ liệu bằng trực quan hóa

Vẽ các đồ thị 2D

Vẽ một số feature theo cặp, tô màu theo Class để xem khả năng phân tách.

```
# Create five 2D plots of the features against each other, color-coded by class.
for x_axis_data, y_axis_data in [
    ('Area', 'Eccentricity'),
    ('Convex_Area', 'Perimeter'),
    ('Major_Axis_Length', 'Minor_Axis_Length'),
    ('Perimeter', 'Extent'),
    ('Eccentricity', 'Major_Axis_Length'),
]:
    px.scatter(rice_dataset, x=x_axis_data, y=y_axis_data, color='Class').show()
```

4 Bài tập 2: Trực quan hóa 3D

Yêu cầu

Thử vẽ 3 feature trong không gian 3D.

```
#@title Plot three features in 3D by entering their names and running this cell

x_axis_data = 'Enter a feature name here' # @param {type: "string"}
y_axis_data = 'Enter a feature name here' # @param {type: "string"}
z_axis_data = 'Enter a feature name here' # @param {type: "string"}
```

```
px.scatter_3d(
    rice_dataset,
    x=x_axis_data,
    y=y_axis_data,
    z=z_axis_data,
    color='Class',
).show()
```

```
# @title One possible solution
```

```
px.scatter_3d(
    rice_dataset,
    x='Eccentricity',
    y='Area',
    z='Major_Axis_Length',
    color='Class',
).show()
```

Nhận xét

Nếu chọn 3 feature, có vẻ `Major_Axis_Length`, `Area` và `Eccentricity` mang nhiều thông tin phân biệt hai lớp. Có thể có các tổ hợp khác cũng hiệu quả. Ta sẽ huấn luyện một mô hình chỉ với 3 feature này, rồi huấn luyện một mô hình dùng **tất cả** feature và so sánh kết quả.

Đặng Tấn Quý

5 Chuẩn hóa dữ liệu (Normalization)

Vì sao cần chuẩn hóa khi có nhiều feature?

Khi mô hình có nhiều feature, các feature nên có thang đo (range) tương đối tương tự. Nếu một feature nằm trong khoảng 500 đến 100,000 còn feature khác chỉ 2 đến 12, mô hình sẽ phải học trọng số cực lớn/cực nhỏ để kết hợp chúng, dẫn đến mô hình kém chất lượng. Để tránh, ta **chuẩn hóa** feature.

Chuẩn hóa theo Z-score

Ta có thể chuẩn hóa bằng Z-score: số độ lệch chuẩn so với trung bình. Ví dụ mean = 60, std = 10:

$$Z = \frac{x - 60}{10}.$$

Khi $x = 75$, $Z = +1.5$. Khi $x = 38$, $Z = -2.2$.

Thực hiện chuẩn hóa cho dataset gạo

Chuyển các cột số sang Z-score và lưu vào DataFrame mới.

```
# Calculate the Z-scores of each numerical column in the raw data and write
# them into a new DataFrame named df_norm.

feature_mean = rice_dataset.mean(numeric_only=True)
feature_std = rice_dataset.std(numeric_only=True)
numerical_features = rice_dataset.select_dtypes('number').columns
normalized_dataset = (
    rice_dataset[numerical_features] - feature_mean
) / feature_std

# Copy the class to the new dataframe
normalized_dataset['Class'] = rice_dataset['Class']

# Examine some of the values of the normalized training set. Notice that most
# Z-scores fall between -2 and +2.
normalized_dataset.head()
```

6 Đặt seed, tạo nhãn và chia train/validation/test

Đặt seed để tái lập thí nghiệm

Để kết quả có thể tái lập, đặt seed cho các bộ sinh số ngẫu nhiên. Nhờ đó thứ tự xáo trộn dữ liệu, giá trị khởi tạo trọng số, ... sẽ giống nhau mỗi lần chạy.

```
keras.utils.set_random_seed(42)
```

Tạo nhãn nhị phân

Ta gán Cammeo nhãn 1 và Osmancik nhãn 0 (tùy ý), rồi huấn luyện bộ phân loại nhị phân.

```
# Create a column setting the Cammeo label to '1' and the Osmancik label to '0'
# then show 10 randomly selected rows.
normalized_dataset['Class_Bool'] = (
    # Returns true if class is Cammeo, and false if class is Osmancik
    normalized_dataset['Class'] == 'Cammeo'
).astype(int)
normalized_dataset.sample(10)
```

Chia tập train/validation/test (80%/10%/10%)

Ta xáo trộn và chia dataset:

- Train: 80%
- Validation: 10%
- Test: 10%

Train dùng để học tham số; validation dùng để so sánh và chọn mô hình; test dùng để báo cáo metric cuối.

```
# Create indices at the 80th and 90th percentiles
number_samples = len(normalized_dataset)
index_80th = round(number_samples * 0.8)
index_90th = index_80th + round(number_samples * 0.1)

# Randomize order and split into train, validation, and test with a .8, .1, .1
split
shuffled_dataset = normalized_dataset.sample(frac=1, random_state=100)
train_data = shuffled_dataset.iloc[0:index_80th]
validation_data = shuffled_dataset.iloc[index_80th:index_90th]
test_data = shuffled_dataset.iloc[index_90th:]

# Show the first five rows of the last split
test_data.head()
```

Tránh rò rỉ nhãn (label leakage)

Trong huấn luyện, không được để nhãn đi vào làm input. Hãy tách feature và label thành các biến riêng.

```
label_columns = ['Class', 'Class_Bool']

train_features = train_data.drop(columns=label_columns)
train_labels = train_data['Class_Bool'].to_numpy()
validation_features = validation_data.drop(columns=label_columns)
validation_labels = validation_data['Class_Bool'].to_numpy()
```

```
test_features = test_data.drop(columns=label_columns)
test_labels = test_data['Class_Bool'].to_numpy()
```

Đặng Tấn Quý

7 Huấn luyện mô hình phân loại nhị phân

7.1 Chọn feature đầu vào

Bắt đầu với 3 feature

Ban đầu, ta huấn luyện với 3 feature: Eccentricity, Major_Axis_Length, Area.

```
# Name of the features we'll train our model on.
input_features = [
    'Eccentricity',
    'Major_Axis_Length',
    'Area',
]
```

7.2 Định nghĩa hàm tạo và huấn luyện mô hình

Ghi chú

Hàm `create_model` bên dưới áp dụng sigmoid để thực hiện **logistic regression**. Ta dùng các cấu trúc `ExperimentSettings` và `Experiment` để lưu lại siêu tham số và kết quả thí nghiệm.

```
# @title Define the functions that create and train a model.

def create_model(
    settings: ml_edu.experiment.ExperimentSettings,
    metrics: list[keras.metrics.Metric],
) -> keras.Model:
    """Create and compile a simple classification model."""
    model_inputs = [
        keras.Input(name=feature, shape=(1,))
        for feature in settings.input_features
    ]

    concatenated_inputs = keras.layers.Concatenate()(model_inputs)
    model_output = keras.layers.Dense(
        units=1, name='dense_layer', activation=keras.activations.sigmoid
    )(concatenated_inputs)
    model = keras.Model(inputs=model_inputs, outputs=model_output)

    model.compile(
        optimizer=keras.optimizers.RMSprop(
            settings.learning_rate
        ),
        loss=keras.losses.BinaryCrossentropy(),
        metrics=metrics,
    )
    return model
```

```
def train_model(
    experiment_name: str,
    model: keras.Model,
    dataset: pd.DataFrame,
    labels: np.ndarray,
    settings: ml_edu.experiment.ExperimentSettings,
) -> ml_edu.experiment.Experiment:
    """Feed a dataset into the model in order to train it."""

    features = {
        feature_name: np.array(dataset[feature_name])
        for feature_name in settings.input_features
    }

    history = model.fit(
        x=features,
        y=labels,
        batch_size=settings.batch_size,
        epochs=settings.number_epochs,
    )

    return ml_edu.experiment.Experiment(
        name=experiment_name,
        settings=settings,
        model=model,
        epochs=history.epoch,
        metrics_history=pd.DataFrame(history.history),
    )

print('Defined the create_model and train_model functions.')
```

7.3 Chạy thí nghiệm baseline và vẽ metric

Metric và threshold

AUC được tính trên nhiều ngưỡng (ở đây 100 ngưỡng), còn accuracy/precision/recall được tính tại một ngưỡng cố định. Trong bài này, threshold ban đầu là 0.35. Hãy thử đổi threshold hoặc learning rate để xem metric thay đổi ra sao.

```
# Let's define our first experiment settings.
settings = ml_edu.experiment.ExperimentSettings(
    learning_rate=0.001,
    number_epochs=60,
    batch_size=100,
    classification_threshold=0.35,
    input_features=input_features,
)
```

```

metrics = [
    keras.metrics.BinaryAccuracy(
        name='accuracy', threshold=settings.classification_threshold
    ),
    keras.metrics.Precision(
        name='precision', thresholds=settings.classification_threshold
    ),
    keras.metrics.Recall(
        name='recall', thresholds=settings.classification_threshold
    ),
    keras.metrics.AUC(num_thresholds=100, name='auc'),
]

# Establish the model's topography.
model = create_model(settings, metrics)

# Train the model on the training set.
experiment = train_model(
    'baseline', model, train_features, train_labels, settings
)

# Plot metrics vs. epochs
ml_edu.results.plot_experiment_metrics(experiment, ['accuracy', 'precision', 'recall'])
ml_edu.results.plot_experiment_metrics(experiment, ['auc'])

```

Vì sao vẽ AUC riêng?

AUC được tính qua toàn bộ ngưỡng (hoặc một tập ngưỡng), còn accuracy/precision/recall chỉ tại 1 ngưỡng. Vì vậy, AUC thường được vẽ tách riêng.

7.4 Đánh giá trên validation

So sánh train và validation

Sau huấn luyện, hãy đánh giá mô hình trên validation và so sánh với train để kiểm tra overfitting.

```

def compare_train_validation(experiment: ml_edu.experiment.Experiment,
    validation_metrics: dict[str, float]):
    print('Comparing metrics between train and validation:')
    for metric, validation_value in validation_metrics.items():
        print('-----')
        print(f'Train {metric}: {experiment.get_final_metric_value(metric):.4f}')
        print(f'Validation {metric}: {validation_value:.4f}')

# Evaluate validation metrics
validation_metrics = experiment.evaluate(validation_features, validation_labels)
compare_train_validation(experiment, validation_metrics)

```

7.5 Huấn luyện mô hình dùng tất cả feature

Có cần tất cả feature không?

Baseline đạt khoảng 90% trên validation. Ta thử mô hình dùng tất cả 7 feature và so sánh AUC.

Bài tập điền feature còn thiếu

Điền các feature còn thiếu vào danh sách dưới đây.

```
# Features used to train the model on.
# Specify all features.
all_input_features = [
    'Eccentricity',
    'Major_Axis_Length',
    'Minor_Axis_Length',
    ? Your code here
]
```

```
#@title Solution
# Features used to train the model on.
# Specify all features.
all_input_features = [
    'Eccentricity',
    'Major_Axis_Length',
    'Minor_Axis_Length',
    'Area',
    'Convex_Area',
    'Perimeter',
    'Extent',
]
```

Huấn luyện mô hình đầy đủ feature và vẽ metric

Đặt threshold = 0.5 và huấn luyện lại.

```
settings_all_features = ml_edu.experiment.ExperimentSettings(
    learning_rate=0.001,
    number_epochs=60,
    batch_size=100,
    classification_threshold=0.5,
    input_features=all_input_features,
)

metrics = [
    keras.metrics.BinaryAccuracy(
        name='accuracy',
        threshold=settings_all_features.classification_threshold,
    ),
    keras.metrics.Precision(
```

```

        name='precision',
        thresholds=settings_all_features.classification_threshold,
    ),
    keras.metrics.Recall(
        name='recall', thresholds=settings_all_features.classification_threshold
    ),
    keras.metrics.AUC(num_thresholds=100, name='auc'),
]

# Establish the model's topography.
model_all_features = create_model(settings_all_features, metrics)

# Train the model on the training set.
experiment_all_features = train_model(
    'all features',
    model_all_features,
    train_features,
    train_labels,
    settings_all_features,
)

# Plot metrics vs. epochs
ml_edu.results.plot_experiment_metrics(
    experiment_all_features, ['accuracy', 'precision', 'recall']
)
ml_edu.results.plot_experiment_metrics(experiment_all_features, ['auc'])

```

7.6 Đánh giá mô hình đầy đủ trên validation

```

validation_metrics_all_features = experiment_all_features.evaluate(
    validation_features,
    validation_labels,
)
compare_train_validation(experiment_all_features, validation_metrics_all_features)

```

Nhận xét (từ notebook)

Mô hình thứ hai có metric train và validation gần nhau hơn, gợi ý rằng nó ít overfit hơn.

7.7 So sánh hai mô hình

```

ml_edu.results.compare_experiment([experiment, experiment_all_features],
    ['accuracy', 'auc'],
    validation_features, validation_labels)

```

Diễn giải kết quả (từ notebook)

Cả hai mô hình có AUC khoảng 0.97–0.98. Không có cải thiện lớn khi thêm 4 feature còn lại, điều này hợp lý vì nhiều feature liên quan chặt chẽ với nhau (ví dụ diện tích, chu

vi, convex area).

Đặng Tấn Quý

8 Tính metric cuối trên test

Vì sao cần tập test?

Để ước lượng chất lượng trên dữ liệu chưa từng thấy, ta tính metric trên tập test sau khi đã chọn mô hình tốt nhất bằng validation. Nếu validation và test chênh lệch lớn, có thể do overfitting hoặc do validation không đại diện phân phối dữ liệu.

```
test_metrics_all_features = experiment_all_features.evaluate(  
    test_features,  
    test_labels,  
)  
for metric, test_value in test_metrics_all_features.items():  
    print(f'Test {metric}: {test_value:.4f}')
```

Kết luận (từ notebook)

Trong bài này, test accuracy khoảng 92%, gần với validation accuracy, nên mô hình kỳ vọng hoạt động tương tự trên dữ liệu mới.